



PROGRAMMING FUNDAMENTALS

CS – 101

Instructor: Maria N. Samad

TUPLES

- A ***tuple*** is a sequence of values
- Values can be of any data type, but indexed by indices starting at 0
- Many operations similar to lists, but the main difference is that Tuples are IMMUTABLE, i.e. once defined they cannot be modified
- Syntactically, it is just a comma-separated list of values:
 - **t1 = ('a', 'b', 'c', 'd', 'e')**
- Tuples enclosed in round brackets, with each value separated by a comma in between

TUPLES

- A tuple can have single value, but to differentiate from variable declaration, it must be ended with a trailing comma if ONLY one value is being defined
 - `t2 = ('a', )`
`print(type(t2))` → #Output ?
- Can create empty tuples using the built-in **tuple** function
 - `t3 = tuple()`
`print(t3)` → #Output ?
- This creates an empty tuple with no initial values

TUPLES

- The **tuple** function also takes in arguments:
 - It takes the argument, breaks it into parts, and gives the output in tuple form, **IFF** the argument can be divided into parts, so applicable to some form of sequences, like lists, strings, and tuples themselves, which can be divided into their elements and be used in the function
 - Example:
 - `str1 = "lupins"`
 - `lst1 = [1.2, 3.4, 5.6, 7.8, 9.0]`
 - `tup1 = ("Hello", 123, '!')`
 - `t1 = tuple(str1)` → Output ?
 - `t2 = tuple(lst1)` → Output ?
 - `t3 = tuple(tup1)` → Output ?

TUPLES

- Most of the list operators are used to access and handle tuples
- Indexing with []
 - **t4 = ('a', 'b', 'c', 'd', 'e')**
print(t4[0]) → **Output ?**
print(t4[5]) → **Output ?**
 - But cannot have them modified, as they are immutable
- Slice Operator
 - Just like strings and lists, slice operator can be used to access part of the tuple
 - **print(t4[1:3])** → **Output ?**
 - Starting index included but not any next

TUPLES

- Tuples cannot be directly modified, but can be replaced using the same variable name to form a new tuple
 - Example:
 - `t4[0] = 'A'`
`print(t4)` → Output ?
 - `t4 = ('A',) + t4[1:]`
`print(t4)` → Output ?

TUPLE ASSIGNMENT

- To assign multiple variables with different values, we need to redefine them on separate lines, but with tuples, we can do it in one line
 - **a, b, c = 1, 2, 3**
- Here values are directly assigned to a, b and c as 1,2, and 3 respectively.
- Can use same variables to do swapping:
 - **a, b = b, a**
- Not limited to two variables, can have as many as you like
- Must have the same number of values as that of variables. If any short, it picks the smaller quantity
- Can apply respective function from the clan, like splitting, slicing, joining etc.

TUPLE ASSIGNMENT

- Example:

- `addr = "monty@python.org"` `#String`
 - `seq = [9, 7, 1, 3, 5]` `#List`
 - `tup1 = (1, 3, 5)` `#Tuple`
 - `tup2 = ('abc',)` `#Tuple`

`uname, domain = addr.split('@')`
`print(uname, domain)` → Output?

`lessthan5, others = seq[:4], seq[4:]`
`print(lessthan5, others)` → Output

`tuple1, tuple2 = tup1, tup2`
`print(tuple1)` → Output ?
`print(tuple2)` → Output ?

- When assigning the values, the original data type of the value like lists, string, etc stays the same even after assignment

TUPLE AS RETURN VALUES

- A function returns a single value
- Tuple is considered as a single, composite entity, so can return multiple values from function in the form of tuples
- Example:
 - `def min_max(x):`
 `return min(x), max(x)`

```
lst2 = [1, 2, 3, 4, 5]    #Can have tuple here as well
mn, mx = min_max(lst2)
print(mn)                 → Output ?
print(mx)                 → Output ?
```

TUPLES AS ARGUMENTS

- If using a tuple as argument, there should be a way to differentiate it from a sequence of multiple arguments
- Uses ***gathers*** and ***scatter*** to achieve that
- When defining a function, the parameters (if tuple) need to be “gathered” together to represent a tuple, and when this function is called, a tuple variable can be sent directly
- When defining a function that needs to use different parts of tuple parameter inside the function body, the parameters need to be gathered for sure, but also when calling the function, the arguments should be sent as “scattered” so that they can be divided into separate values, as represented by the function body

TUPLES AS ARGUMENTS

- Example:

- **def printall(*args):**
 print(args)

printall(1, 2.0, 'a') **→ Output ?**

printall((1, 2.0, 'a'),) **→ Output ?**

t5 = (1, 2.0, 'a')
printall(t5) **→ Output ?**

TUPLES AS ARGUMENTS

- Same function can then be used for other sequences like strings and lists, as it will convert the sequence into tuples
- Example:
 - `lst2 = ['h', 'e', 'l', 'l', 'o']`
`printall(lst2)` → Output ?
 - `str2 = "Hello World"`
`printall(str2)` → Output ?

TUPLES AS ARGUMENTS

- When individual members of tuples need to be accessed, we must define the function with asterisk and parameter list name, as well as call it with that way too

- Example:

- **def quotrem(*args):**
 quot = args[0]//args[1]
 rem = args[0] % args[1]
 return quot, rem

t6 = (7,3)

q, r = quotrem(*t6)

print(q, r)

→ Output ?

LISTS AND TUPLES

- Python provides a built-in function to derive a list of tuples by combining two or more sequences (strings and/or lists)
- It combines it in such a way that the list of tuples then contain one element from each sequence chosen in the same order as defined
- If the length of the sequences is not the same, it will pick the shortest length as the length of the list of tuples
- Use ***zip*** function to do that
- Example:
 - `str3 = "abc"`
`lst3 = [0,1,2]`
`tup_it = zip(str3,lst3)`
`lst_tup = list(tup_it)`
`print(lst_tup)` → Output ?

LISTS AND TUPLES

- Can use tuples as iterators, which are essentially a list too
- Example:
 - **for letter, number in lst_tup:**
 print(number, letter) **→ Output ?**

DICTIONARIES AND TUPLES

- Python provides another built-in function for dictionaries that returns a list of tuples, but now the tuples are key-value pairs of dictionary, so only one sequence involved, i.e. a dictionary
- Use ***items()*** function to get that list
- Just like dictionaries, this list of items may not have a fixed order
- Example:
 - `dict1 = {'f': 4, 'r': 2, 's': 6, 'a': 1}`
`dict_lst_tup = dict1.items()`
`print(dict_lst_tup)` → Output ?

DICTIONARIES AND TUPLES

- The ***dict*** function can use tuples to form a new dictionary, i.e. the opposite of what ***items*** achieved
- Example:
 - `dict2 = dict(dict_lst_tup)`
`print(dict2)` → Output ?
- Can combine both ***zip*** function of the lists and ***dict*** function of the dictionaries to get a dictionary from tuples in one step
- Example:
 - `dict3 = dict(zip("abc", range(3)))`
`print(dict3)` → Output ?

DICTIONARIES AND TUPLES

- The ***update*** function of dictionaries can be used with tuples to add a list of them to an existing dictionary
- Example:
 - `lst_tups = list(zip("xyz", (1,2,3)))`
`dict3.update(lst_tups)`
`print(dict3)` → Output ?

DICTIONARIES AND TUPLES

- Combining various aspects of dictionaries and tuples, we can define a loop to traverse the keys and values of dictionaries together in the same definition of the loop
- Example:
 - **for key, val in dict3.items():
 print(val, key)**

DICTIONARIES AND TUPLES

- Tuples are immutable, so can be used as keys in dictionaries, unlike lists, which can't
- Hence we can define a dictionary with a tuple containing multiple variables as it's key and then use whatever data type for the value

- Example:

- `dict4 = dict()`
`dict4["Cleese", "John"] = 123`
`dict4["Chapman", "Graham"] = 234`
`dict4["Cleese", "Harry"] = 345`
`dict4["Jones", "Terry"] = 456`
`dict4["Idle", "Eric"] = 567`
`dict4["Chapman", "Charles"] = 678`

`print(dict4)`

→ Output ?

DICTIONARIES AND TUPLES

- To access the individual keys-value pairs, we can define a for loop with both the key attributes of a tuple
- Example:
 - **for last_name, first_name in dict4:**
 print(first_name, last_name, dict4[last_name, first_name]) → Output ?

COMPARING TUPLES

- Can use relational operators with tuples
 - == != < >
- Except for equality and inequality, the remaining operators returns True or False on the first comparison, without checking all the remaining variables of tuples

- Example:

- `t7 = (0, 1, 2)`
`t8 = (0, 3, 1)`
`t9 = (0, 1, 2)`

<code>print(t7 < t8)</code>	→ Output ?
<code>print(t7 == t8)</code>	→ Output ?
<code>print(t7 > t8)</code>	→ Output ?
<code>print(t7 != t8)</code>	→ Output ?

<code>print(t7 < t9)</code>	→ Output ?
<code>print(t7 == t9)</code>	→ Output ?
<code>print(t7 > t9)</code>	→ Output ?
<code>print(t7 != t9)</code>	→ Output ?